

UNIVERSIDADE FEDERAL DE SERGIPE (UFS)
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA (CCET)
DEPARTAMENTO DE COMPUTAÇÃO (DCOMP)
ENGENHARIA DE SOFTWARE – COMP0503

Descrição Arquitetural do Sistema de Intermediação de Brechós

Conforme ISO/IEC/IEEE 42010:2022

Caroline Santos, Beatriz Eduão, Ian Daniel
João Pedro M., Arthur Soares, Luiz Manoel

Prof. Dr. Michel Soares

Aracaju – SE
23 de junho de 2026

Sumário

1	Identificação da Architecture Description	2
1.1	Entity of Interest	2
1.2	Escopo da Architecture Description	2
1.3	Ambiente (<i>Environment</i>)	2
1.3.1	Influências Técnicas	2
1.3.2	Influências Operacionais	3
1.3.3	Influências Regulatórias e Legais	3
1.3.4	Influências Organizacionais	3
1.4	Referências Normativas	3
1.5	Glossário	3
2	Stakeholders e Concerns	4
2.1	Identificação dos Stakeholders	4
2.2	Identificação dos Concerns	5
2.2.1	Concerns de Qualidade (Atributos de Qualidade)	5
2.2.2	Concerns Contextuais	7
2.3	Matriz Concern $\times$ Stakeholder	8
3	Architecture Viewpoints	9
3.1	VP1 — Viewpoint Lógico	9
3.2	VP2 — Viewpoint de Desenvolvimento	10
3.3	VP3 — Viewpoint de Processos	10
3.4	VP4 — Viewpoint de Implantação	11
3.5	Viewpoint de Cenários (+1)	11
3.6	Rastreabilidade Concern $\rightarrow$ Viewpoint	12
4	Architecture Views	12
4.1	View Lógica (VP1)	12
4.2	View de Desenvolvimento (VP2)	14
4.3	View de Implantação (VP4)	15
4.4	View de Cenários (+1)	16
5	Architecture Decisions e Rationale	17
5.1	Decisão 01: Estilo Arquitetural em Camadas (Arquitetura Limpa) e Servidor Stateless	17
5.2	Decisão 02: Padrão de Persistência com Data Mapper	18
5.3	Decisão 03: Middleware Orientado a Mensagens (MOM) via WebSockets	18
5.4	Decisão 04: Stack Tecnológica de Implementação (Go e React)	19
6	Correspondences e Correspondence Rules	19
6.1	Regras de Correspondência (Correspondence Rules)	19
6.2	Correspondências Identificadas (Correspondences)	20
6.2.1	Mapeamento de Concerns para Decisões Arquiteturais (Aplica CR-02)	20
6.2.2	Rastreabilidade Lógica vs. Física de Implantação (Aplica CR-01)	21
6.2.3	Matriz de Restrição de Dependência de Camadas (Aplica CR-03)	21

1 Identificação da Architecture Description

Este documento constitui a *Architecture Description* (AD) do Sistema de Intermediação de Brechós, elaborada em conformidade com a norma ISO/IEC/IEEE 42010:2022.

1.1 Entity of Interest

A entidade de interesse (*Entity of Interest*) desta AD é o **Sistema de Intermediação de Brechós** — uma plataforma web no modelo C2C (*Consumer-to-Consumer*) destinada à compra e venda de peças de vestuário usadas. O sistema permite que usuários atuem simultaneamente como compradores e vendedores, intermediando o processo de anúncio, busca, comunicação, pagamento e acompanhamento de pedidos.

O sistema é classificado como uma aplicação web de domínio comercial, independente e totalmente contida.

1.2 Escopo da Architecture Description

Esta AD cobre a arquitetura de software do sistema em sua totalidade, abrangendo:

- a organização dos componentes de software e suas interfaces;
- as decisões arquiteturais significativas e suas justificativas;
- o mapeamento dos componentes nos ambientes de execução;
- as táticas arquiteturais adotadas para atender aos atributos de qualidade.

Esta AD **não** cobre: o design detalhado de interfaces gráficas (UI/UX) nem a implementação interna dos serviços externos integrados (código-fonte do Stripe, Amazon S3 ou gateway de logística). A forma de integração com esses serviços (protocolos, padrões de comunicação e contratos de API), bem como a infraestrutura *serverless* utilizada para hospedagem na plataforma Vercel (Plano Hobby), fazem parte do escopo desta AD.

1.3 Ambiente (*Environment*)

Conforme a ISO/IEC/IEEE 42010:2022, o ambiente compreende todas as influências sobre a entidade de interesse. O Sistema de Intermediação de Brechós opera sob as seguintes influências:

1.3.1 Influências Técnicas

- Plataforma web acessível via navegadores modernos: Chrome (v147+), Firefox (v150+), Safari (v26+) e Microsoft Edge (v148+).
- Dependência de conectividade à internet para todas as operações.
- Hospedagem da camada de apresentação (SPA) e da API central em plataforma PaaS/*Serverless* Vercel (Plano Hobby), com CDN integrada (*Vercel Edge Network*).
- Integração com serviços externos: gateway de pagamentos (API Stripe v1, REST/HTTPS), armazenamento de mídia (Amazon S3, API REST v2) e banco de dados relacional (PostgreSQL 15+).
- Comunicação cliente-servidor via HTTPS; comunicação em tempo real via WebSocket (chat) através do serviço PaaS Render; notificações via SMTP.

- Stack tecnológica de implementação definida para o ciclo atual: *framework* React (TypeScript/JavaScript) para a camada de Apresentação (SPA) e linguagem Go (Golang) para o *backend*, operando sobre o modelo *serverless*.

1.3.2 Influências Operacionais

- O sistema deve suportar acesso contínuo de múltiplos usuários simultâneos, com exceção de períodos de manutenção programada.
- Adaptação exclusiva ao idioma português brasileiro (PT-BR), moeda Real (R\$) e formatos locais de data e endereço.
- Upload de imagens limitado a 5MB por arquivo e 5 imagens por anúncio; paginação obrigatória de listagens extensas em pacotes de 20 a 30 registros.

1.3.3 Influências Regulatórias e Legais

- Lei Geral de Proteção de Dados — LGPD (Lei nº 13.709/2018): armazenamento e tratamento de dados pessoais e sensíveis.
- Código de Defesa do Consumidor — CDC (Lei nº 8.078/1990): transparência e responsabilidade nas relações de consumo.
- Marco Civil da Internet (Lei nº 12.965/2014): proteção dos direitos dos usuários no ambiente digital.
- Presença obrigatória de Termos de Uso e Políticas de Privacidade na plataforma.

1.3.4 Influências Organizacionais

- Projeto desenvolvido no âmbito da disciplina COMP0503 — Engenharia de Software, Universidade Federal de Sergipe, sob orientação do Prof. Dr. Michel Soares.
- Equipe composta por 6 desenvolvedores com restrições de cronograma acadêmico.
- Padronização de código-fonte e documentação atualizada como requisitos organizacionais.

1.4 Referências Normativas

- ISO/IEC/IEEE 42010:2022 — *Software, systems and enterprise — Architecture description*.
- IEEE Std 830-1998 — *IEEE Recommended Practice for Software Requirements Specifications*.
- Bass, L.; Clements, P.; Kazman, R. *Software Architecture in Practice*. 3rd ed. Addison-Wesley, 2012.
- Kruchten, P. The 4+1 View Model of Architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, 1995.
- Clements, P. et al. *Documenting Software Architectures: Views and Beyond*. 2nd ed. Addison-Wesley, 2010.

1.5 Glossário

Termo	Definição
Architecture Description (AD)	Artefato utilizado para expressar a arquitetura de uma entidade de interesse, conforme ISO/IEC/IEEE 42010:2022.
Entity of Interest	Entidade cuja arquitetura está sendo descrita — neste caso, o Sistema de Intermediação de Brechós.
Stakeholder	Indivíduo, equipe ou organização com interesses no sistema.
Concern	Qualquer interesse no sistema: expectativa, necessidade, objetivo, restrição, atributo de qualidade, risco.
Architecture Viewpoint	Conjunto de convenções para construir, interpretar e analisar um tipo de Architecture View.
Architecture View	Representação da arquitetura sob a perspectiva de um conjunto de concerns, governada por viewpoint.
Model Kind	Convenções para um tipo de modelo arquitetural (linguagem, notação, técnicas analíticas).
Architecture Decision	Decisão de design significativa que afeta <i>elements</i> da AD e pertence a um ou mais concerns.
Architecture Rationale	Justificativa, explicação ou raciocínio sobre decisões arquiteturais tomadas e alternativas descartadas.
Correspondence	Relação entre elementos da AD (rastreadibilidade, consistência, refinamento).
Correspondence Rule	Regra que governa e verifica correspondências dentro ou entre ADs.
C2C	<i>Consumer-to-Consumer</i> — modelo de comércio eletrônico entre consumidores.
SRS	<i>Software Requirements Specification</i> — Especificação de Requisitos de Software.

2 Stakeholders e Concerns

Conforme a ISO/IEC/IEEE 42010:2022, uma Architecture Description deve identificar os stakeholders cujas preocupações são consideradas fundamentais para a arquitetura, bem como os concerns associados a cada um deles. Esta seção apresenta os stakeholders do Sistema de Intermediação de Brechós e os concerns arquiteturalmente significativos, com ênfase nos atributos de qualidade de desempenho, escalabilidade, segurança e disponibilidade.

2.1 Identificação dos Stakeholders

Stakeholder	Papel (42010)	Descrição
Comprador	<i>User</i>	Usuário que pesquisa, filtra e adquire peças de vestuário na plataforma. Interage com o sistema por meio do feed de anúncios, carrinho de compras, chat e acompanhamento de pedidos.
Vendedor	<i>User</i>	Usuário que publica anúncios, gerencia itens e realiza envios. Recebe repasses financeiros após a confirmação de entrega. Um usuário pode exercer ambos os papéis (comprador e vendedor) simultaneamente.
Equipe de Desenvolvimento	<i>Developer, Builder, Maintainer</i>	Responsável pela implementação, testes e manutenção do código-fonte do sistema.
Equipe de Infraestrutura	<i>Operator</i>	Responsável pela implantação, operação e monitoramento do sistema em ambiente de produção, incluindo a infraestrutura em nuvem.
Product Owner	<i>Acquirer, Owner</i>	Responsável por avaliar os resultados. No contexto deste projeto, esse papel é exercido pelo Prof. Dr. Michel Soares.

2.2 Identificação dos Concerns

A ISO/IEC/IEEE 42010:2022 define *concern* como qualquer interesse no sistema — expectativas, necessidades, objetivos, restrições, atributos de qualidade ou riscos.

Os concerns identificados para esta AD estão organizados em duas categorias: concerns de qualidade priorizados e concerns contextuais.

2.2.1 Concerns de Qualidade (Atributos de Qualidade)

Estes são os concerns prioritários desta Architecture Description.

CQ01 — Desempenho

O desempenho diz respeito ao volume de trabalho que o sistema deve executar em um período de tempo e aos limites de tempo de resposta que devem ser respeitados. No Sistema de Intermediação de Brechós, o desempenho é crítico em três pontos: a renderização do feed de anúncios, o processamento de transações de compra e a latência do chat em tempo real.

Métrica	Especificação
Tempo de resposta (Response Time)	95% das requisições HTTP devem ser respondidas pelo servidor em até 500ms. Nenhuma requisição deve exceder 3 segundos.
Throughput	O sistema deve processar no mínimo 100 requisições por segundo no pico de utilização, considerando operações de leitura (feed, busca) e escrita (criação de anúncios, compras).

Latência do chat	Mensagens enviadas pelo chat devem ser entregues ao destinatário em até 200ms via WebSocket.
Renderização inicial	O feed de anúncios deve ser renderizado no navegador do usuário em até 3 segundos na primeira carga.

As táticas arquiteturais adotadas para atender a este concern são detalhadas na Seção 5 (Architecture Decisions e Rationale).

CQ02 — Escalabilidade

A escalabilidade refere-se à capacidade do sistema de manter seu desempenho quando a demanda aumenta, seja em volume de requisições, conexões simultâneas ou tamanho dos dados armazenados. No paradigma *serverless* adotado pela Vercel, a plataforma escala de forma transparente e instantânea de zero a milhares de funções concorrentes por requisição, sem necessidade de provisionamento de servidores. O limite de escalabilidade passa a ser medido pelas cotas do Plano Hobby: 1 milhão de invocações de funções por mês e 100 GB de transferência de dados.

Dimensão	Decisão Arquitetural
Elasticidade <i>serverless</i>	A Vercel instancia funções efêmeras e isoladas sob demanda para cada requisição HTTP. A arquitetura é inerentemente <i>stateless</i> , dispensando provisionamento manual de instâncias. O estado de sessão é mantido via tokens JWT armazenados no cliente, já que não há memória compartilhada entre invocações.
Carga de requisições	Operações de leitura intensiva (feed de anúncios, busca e filtragem) devem utilizar caching para reduzir a carga sobre o banco de dados. A paginação de resultados (20 a 30 registros por requisição) limita o volume de dados trafegados por chamada.
Volume de dados	As consultas ao banco de dados devem utilizar índices nos campos de filtragem e busca (categoria, tamanho, faixa de preço, status do item). Arquivos de mídia (imagens de anúncios) são armazenados em serviço externo (Amazon S3), evitando crescimento do banco de dados relacional.

As táticas arquiteturais adotadas para viabilizar a escalabilidade são detalhadas na Seção 5 (Architecture Decisions e Rationale).

CQ03 — Segurança

A segurança abrange a capacidade do sistema de proteger dados e recursos contra acessos não autorizados, garantindo autenticação, autorização e criptografia.

Métrica	Especificação
Autenticação	O sistema deve verificar a identidade dos usuários por meio de credenciais (e-mail/CPF e senha). Senhas devem ser armazenadas com hash bcrypt (custo mínimo 10).

Autorização	Usuários autenticados devem ter acesso restrito às operações permitidas ao seu papel. Um vendedor só pode editar ou excluir anúncios próprios. Um comprador só pode acessar pedidos próprios.
Criptografia	Todas as comunicações cliente-servidor devem utilizar TLS 1.2 ou superior. Dados sensíveis de pagamento não devem ser armazenados no sistema — o processamento financeiro é delegado integralmente ao gateway externo (Stripe).
Conformidade LGPD	O sistema deve permitir que o usuário solicite a exclusão de seus dados pessoais. Dados sensíveis (CPF, endereço, dados bancários) devem ser armazenados com criptografia em repouso.
Tokens de sessão	Tokens de autenticação devem ter expiração máxima de 24 horas e serem invalidados no logout.

As táticas arquiteturais adotadas para atender a este concern são detalhadas na Seção 5 (Architecture Decisions e Rationale).

CQ04 — Disponibilidade

A disponibilidade refere-se à probabilidade de que o sistema esteja pronto para executar suas tarefas quando o usuário necessitar. Devido à escolha de infraestrutura gratuita (CC02), a arquitetura não pode garantir SLAs de *uptime* de 99,99% (como em provedores AWS *Enterprise*). Portanto, a abordagem arquitetural foca em minimizar o MTTR (*Mean Time to Repair*) e em aplicar táticas de tolerância a falhas.

Métrica / Tática	Especificação Aplicada
Recuperação de Falhas (Baixo MTTR)	O uso de <i>Serverless Functions</i> (Vercel) garante que, caso uma instância de execução trave ou falhe, a próxima requisição instanciará uma função totalmente nova e limpa. O MTTR lógico da API é de milissegundos.
Recuperação de Falhas (Degradação Graciosa)	Tática aplicada no <i>Chat</i> . Em caso de queda do <i>middleware</i> de WebSocket (Render), o sistema não trava a tela do usuário. Ele enfileira mensagens silenciosamente (<i>local storage</i>) e aguarda a reconexão.
Recuperação de Falhas (<i>Rollback</i> e Transações)	Tática aplicada aos pagamentos. Se a API externa do Stripe falhar ou apresentar <i>timeout</i> , o sistema reverte a operação localmente (<i>rollback</i> no PostgreSQL) e libera a peça, impedindo que a falha do serviço externo corrompa o banco de dados interno.

As táticas arquiteturais adotadas para atender a este concern são detalhadas na Seção 5 (Architecture Decisions e Rationale).

2.2.2 Concerns Contextuais

ID	Concern	Descrição
CC01	Propósito do sistema	Intermediar de forma organizada e segura o processo de compra e venda de peças de vestuário usadas entre consumidores, promovendo os brechós como experiência digital intuitiva.
CC02	Viabilidade de construção e implantação	O sistema é desenvolvido por uma equipe de 6 integrantes em contexto acadêmico, com restrições de cronograma e uso do plano gratuito da Vercel para hospedagem. Essas restrições influenciam diretamente as decisões arquiteturais, em particular os limites de escalabilidade e disponibilidade.
CC03	Manutenibilidade e evolvibilidade	O sistema deve ser estruturado de forma que permita a adição de novas funcionalidades e a correção de defeitos com impacto controlado sobre os demais componentes. Embora não seja um atributo de qualidade priorizado nesta AD, a organização em camadas e o baixo acoplamento entre componentes favorecem a manutenibilidade.
CC04	Riscos e impactos ao longo do ciclo de vida	Os principais riscos identificados incluem: indisponibilidade de serviços externos (Stripe, Amazon S3), esgotamento dos limites do plano gratuito da Vercel e exposição de dados pessoais em desconformidade com a LGPD.

2.3 Matriz Concern $\times$ Stakeholder

Concern	<i>Comprador</i>	<i>Vendedor</i>	<i>Eq. Desenvolvimento</i>	<i>Eq. Infraestrutura</i>	<i>Product Owner</i>
CQ01 – Desempenho	•	•	•	•	•
CQ02 – Escalabilidade				•	•
CQ03 – Segurança	•	•	•	•	•
CQ04 – Disponibilidade	•	•		•	•
CC01 – Propósito	•	•			•
CC02 – Viabilidade			•	•	•

Concern	Comprador	Vendedor	Eq. Desenvolvimento	Eq. Infraestrutura	Product Owner
CC03 – Manutenibilidade			•		•
CC04 – Riscos	•	•	•	•	•

3 Architecture Viewpoints

Conforme a ISO/IEC/IEEE 42010:2022, um *Architecture Viewpoint* é um conjunto de convenções para construir, interpretar, utilizar e analisar um tipo de *Architecture View*. Cada viewpoint enquadra (*frames*) um conjunto específico de concerns. Esta AD adota quatro viewpoints fundamentais e um integrado (+1), mantendo o escopo estritamente focado em elements exclusivos da arquitetura global.

3.1 VP1 — Viewpoint Lógico

Nome	Viewpoint Lógico
Concerns enquadrados	CC01 (Propósito do sistema), CQ03 (Segurança — regras de autorização)
Stakeholders interessados	Comprador, Vendedor, Equipe de Desenvolvimento, Product Owner
Model Kinds próprios	MK1 — Modelo Conceitual de Domínio
Referências	Kruchten (1995), Bass et al. (2012), UML 2.5 Specification

MK1 — Modelo Conceitual de Domínio

Linguagem/Notação	UML 2.5 — Diagrama de Classes (Alto Nível)
Elementos utilizados	Entidades conceituais («class»), associações, composições e multiplicidades. (Ocultam-se atributos, enumerações e métodos para preservar a abstração arquitetural).
Ferramenta	Draw.io ou Astah
Propósito	Representar a estrutura conceitual do domínio do sistema, identificando as entidades e os relacionamentos semânticos que sustentam as regras de negócio da intermediação C2C, sem viés de implementação.

Referências Externas Complementares: O projeto não utiliza a modelagem Entidade-Relacionamento (ER) clássica. A definição do esquema físico de banco de dados e as

amarrações de chaves no PostgreSQL são tratadas exclusivamente como artefatos de *Design de Software* (nível de implementação), através do Mapeamento Objeto-Relacional (ORM). Portanto, esses detalhes técnicos foram intencionalmente abstraídos desta Descrição Arquitetural para focar puramente na estrutura conceitual de alto nível.

3.2 VP2 — Viewpoint de Desenvolvimento

Nome	Viewpoint de Desenvolvimento
Concerns enquadra- dos	CC03 (Manutenibilidade e evolvabilidade), CC02 (Viabilidade de construção)
Stakeholders interes- sados	Equipe de Desenvolvimento
Model Kinds próprios	MK2 — Diagrama de Camadas
Referências	Parnas (1972), Dijkstra (1968), Bass et al. (2012)

MK2 — Diagrama de Camadas

Linguagem/Notação	UML 2.5 — Diagrama de Pacotes, organizado em camadas horizontais com restrições de visibilidade
Elementos utilizados	Pacotes («package»), dependências direcionadas, estereótipos de camada
Ferramenta	Draw.io ou Astah
Propósito	Representar a organização do código-fonte em camadas (Apresentação, Aplicação, Domínio, Infraestrutura) e as restrições de dependência entre elas

Referências Externas Complementares: Os padrões arquiteturais em formato de texto e os guias de estilo de implementação estão concentrados e documentados na **Seção 5 (Architecture Decisions e Rationale)** deste documento, eliminando sobreposições com os artefatos de código.

3.3 VP3 — Viewpoint de Processos

Nome	Viewpoint de Processos
Concerns enquadra- dos	CQ01 (Desempenho — fluxos de resposta), CQ03 (Segurança — fluxo de autenticação), CQ04 (Disponibilidade — tratamento de falhas)
Stakeholders interes- sados	Comprador, Vendedor, Equipe de Desenvolvimento, Equipe de Infraestrutura
Model Kinds próprios	Nenhum (Utiliza apenas diagramas externos de referência)
Referências	Bass et al. (2012), Kruchten (1995)

Referências Externas Complementares: A concorrência de funções *serverless* (Vercel) e o tráfego do *middleware* assíncrono (WebSockets) compõem a Visão de Processos. Não foram criados Model Kinds proprietários nesta seção, uma vez que a dinâmica de mensageria e o fluxo temporal de estados encontram-se representados nos diagramas complementares (Máquina de Estados) gerados para avaliação do projeto de software.

3.4 VP4 — Viewpoint de Implantação

Nome	Viewpoint de Implantação
Concerns enquadrados	CQ01 (Desempenho — topologia de infraestrutura), CQ02 (Escalabilidade), CQ04 (Disponibilidade — redundancy e monitoramento), CC02 (Viabilidade — restrições do plano de hospedagem), CC04 (Riscos — dependência de serviços externos)
Stakeholders interessados	Equipe de Infraestrutura, Equipe de Desenvolvimento, Product Owner
Model Kinds próprios	MK3 — Diagrama de Implantação
Referências	Bass et al. (2012), Clements et al. (2010)

MK3 — Diagrama de Implantação

Linguagem/Notação	UML 2.5 — Diagrama de Implantação (<i>Deployment Diagram</i>)
Elementos utilizados	Nós de execução («device», «executionEnvironment»), artefatos, associações de comunicação, protocolos
Ferramenta	Draw.io ou Astah
Propósito	Representar o mapeamento dos componentes de software (do VP1) nos nós de hardware e serviços, mostrando onde cada componente é executado e como os nós se comunicam

Referências Externas Complementares: A representação técnica e a visualização física de infraestrutura foram agregadas e consolidadas junto ao escopo do MK3. A plataforma de hospedagem adotada é a Vercel, que provê CDN integrada, *Serverless Functions* e integração nativa com repositórios Git.

3.5 Viewpoint de Cenários (+1)

Nome	Viewpoint de Cenários (Casos de Uso)
Concerns enquadrados	CC01 (Propósito do sistema), CQ01 (Desempenho), CQ04 (Disponibilidade)
Stakeholders interessados	Comprador, Vendedor, Equipe de Desenvolvimento, Product Owner
Model Kinds próprios	Nenhum (Utiliza Diagramas UML externos à AD)
Referências	Kruchten (1995)

Referências Externas Complementares: Em conformidade com o modelo 4+1, esta visão atua como o elo que unifica as quatro visões anteriores, ilustrando como os elementos da arquitetura trabalham juntos. A validação destes fluxos baseia-se em diagramas dinâmicos gerados e mantidos no documento externo de modelagem UML da equipe:

- **Diagrama de Sequência:** Fornece a rastreabilidade temporal de trocas de mensagens, exemplificando o comportamento da API Central durante transações.

- **Diagrama de Atividades:** Mapeia os cenários de interação e o fluxo de controle transversal entre usuários, o sistema e o *Gateway* de Pagamento.

3.6 Rastreabilidade Concern $\rightarrow$ Viewpoint

Concern	Viewpoint(s) que o enquadra(m)
CQ01 – Desempenho	VP3 (Processos), VP4 (Implantação), Cenários (+1)
CQ02 – Escalabilidade	VP4 (Implantação)
CQ03 – Segurança	VP1 (Lógico), VP3 (Processos)
CQ04 – Disponibilidade	VP3 (Processos), VP4 (Implantação), Cenários (+1)
CC01 – Propósito	VP1 (Lógico), Cenários (+1)
CC02 – Viabilidade	VP2 (Desenvolvimento), VP4 (Implantação)
CC03 – Manutenibilidade	VP2 (Desenvolvimento)
CC04 – Riscos	VP4 (Implantação)

4 Architecture Views

Esta seção instancia os modelos estruturais definidos nos *Architecture Viewpoints* Lógico, de Desenvolvimento, de Implantação e de Cenários (Seção 3), apresentando as representações visuais da arquitetura do Sistema de Intermediação de Brechós em escala ampliada e sem rotação.

4.1 View Lógica (VP1)

A View Lógica apresenta o Modelo Conceitual de Domínio do sistema. Este diagrama abstrai os detalhes de implementação de código (métodos e classes de infraestrutura) para focar exclusivamente nas entidades centrais do negócio de intermediação de brechós (C2C) e seus relacionamentos. Este modelo serve como base para a camada de Domínio, orientando o padrão *Data Mapper* adotado na arquitetura.

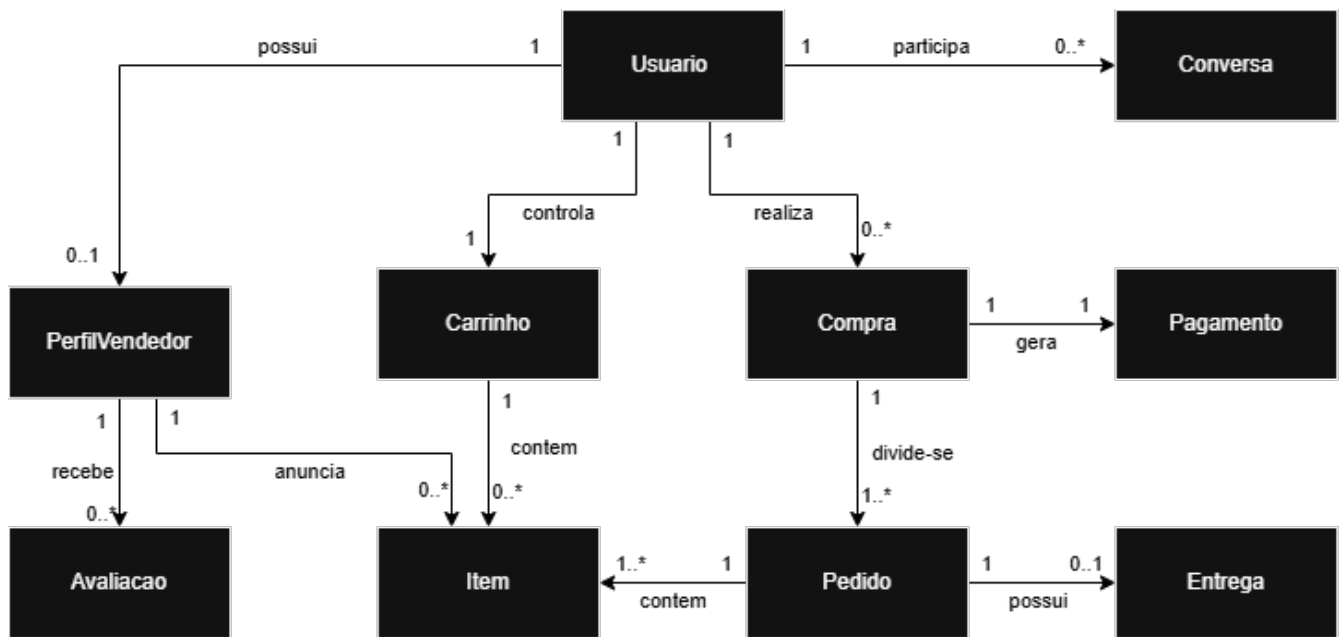


Figura 1: Modelo Conceitual de Domínio do Sistema de Brechós (MK1 — VP1)

4.2 View de Desenvolvimento (VP2)

A View de Desenvolvimento detalha a organização lógica do código-fonte estruturada em camadas horizontais bem definidas, conforme as restrições de arquitetura e isolamento de dependências. O Diagrama de Camadas (MK2) explicita os limites de visibilidade direcionais, garantindo o completo isolamento das regras de negócio contidas no núcleo do Domínio em relação às tecnologias de Infraestrutura e Apresentação.

Na materialização física destas camadas, a camada de Apresentação é implementada utilizando o ecossistema React, empacotada como uma *Single Page Application* (SPA). As camadas subjacentes (Aplicação, Domínio e Infraestrutura) são implementadas na linguagem Go, garantindo forte tipagem e alocação eficiente de memória para as funções *serverless* que encapsulam as regras C2C.

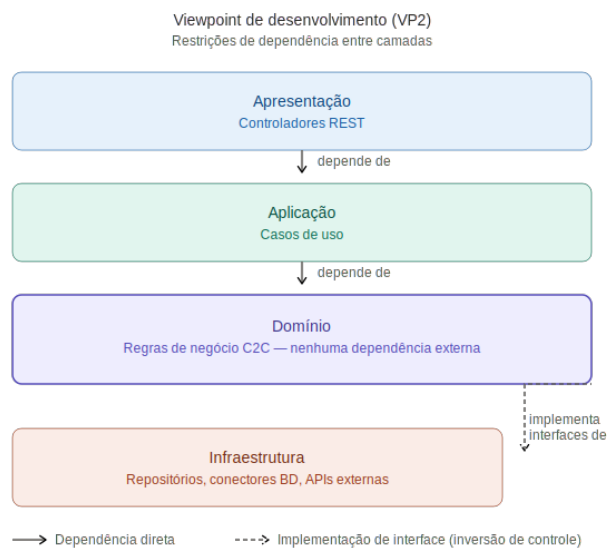


Figura 2: Diagrama de Camadas do Sistema de Brechós (MK2 — VP2)

4.3 View de Implantação (VP4)

A View de Implantação expõe o mapeamento físico e lógico dos artefatos estruturais do software nos respectivos nós computacionais de execução. Diferente de uma arquitetura *three-tier* tradicional, a infraestrutura adota uma **Topologia Distribuída em Nuvem (Cloud-Native)**, aproveitando serviços gerenciados (PaaS, DBaaS e *Serverless*) para maximizar a escalabilidade e o isolamento de falhas.

O Diagrama de Implantação (MK3) ilustra este mapeamento: a camada de Apresentação é entregue via CDN (*Vercel Edge Network*) e executada no navegador do cliente; a lógica de Aplicação é paralelizada entre *Serverless Functions* (Vercel) e um contêiner PaaS dedicado (Render) para conexões persistentes; e o Gerenciamento de Recursos é fragmentado em serviços especializados de banco de dados (*Neon Postgres*) e armazenamento de objetos (*Amazon S3*), integrando-se externamente ao *Stripe*. Esta distribuição atende rigorosamente à regra de completude CR-01.

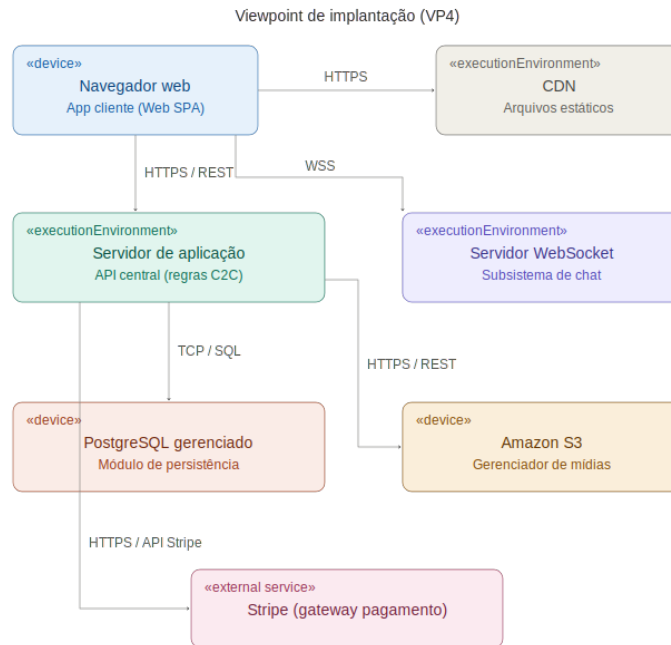


Figura 3: Diagrama de Implantação de Infraestrutura Serverless (MK3 — VP4)

4.4 View de Cenários (+1)

A View de Cenários (+1) consolida e valida a arquitetura, demonstrando como os elementos lógicos, de processo e físicos colaboram para entregar as funcionalidades centrais do sistema. A ilustração e validação dos cenários críticos do negócio C2C (como a publicação de um anúncio pelo vendedor e o fluxo transversal de compra) são realizadas por meio de Diagramas de Atividades e de Sequência.

Devido ao seu alto nível de detalhamento de interações, assinaturas de métodos e fluxo de controle estrito, **estes diagramas compõem formalmente o Documento de Design UML (anexo externo à AD)**, atuando como o elo prático entre esta abstração arquitetural e a implementação técnica das funcionalidades.

5 Architecture Decisions e Rationale

Esta seção apresenta as decisões arquiteturais significativas tomadas para o Sistema de Intermediação de Brechós, justificando-as com base nos conceitos de táticas arquiteturais e nos cenários de atributos de qualidade formulados na Seção 2.

5.1 Decisão 01: Estilo Arquitetural em Camadas (Arquitetura Limpa) e Servidor Stateless

Decisão: O sistema adota uma organização lógica estruturada em quatro camadas horizontais rígidas (Apresentação, Aplicação, Domínio e Infraestrutura) sob a ótica da **Arquitetura Limpa (Clean Architecture)**, empregando o Princípio da Inversão de Dependência, combinada com um modelo de comunicação *stateless* (sem estado) no servidor de aplicação.

Concerns Enquadrados: CC03 (Manutenibilidade), CQ02 (Escalabilidade) e CQ04 (Disponibilidade).

Rationale (Justificativa):

- **Uso de Camadas e Arquitetura Limpa (CC03):** A imposição de restrições de visibilidade estritas entre as camadas isola as regras de negócio das tecnologias de infraestrutura. A adoção da Inversão de Controle força a Infraestrutura a depender do Domínio, protegendo o núcleo do negócio de mudanças em APIs externas e bancos de dados, alinhando-se aos princípios mais modernos de isolamento. Isso reduz o acoplamento global, permitindo modificações isoladas em componentes específicos (como a troca de bibliotecas na camada de Apresentação) sem causar efeitos colaterais na lógica de domínio, o que atende diretamente ao interesse de manutenibilidade da equipe de desenvolvimento.
- **Estruturação Stateless (CQ02 e CQ04):** Na plataforma Vercel, cada requisição HTTP acorda uma função *serverless* isolada e efêmera, sem memória compartilhada entre invocações. Essa característica torna obrigatório o uso de tokens JWT autocontidos armazenados no cliente para manter o estado da sessão. Como nenhuma função retém estado, a Vercel pode escalar transparentemente o número de funções concorrentes conforme a demanda, sem necessidade de provisionamento manual de instâncias ou configuração de balanceadores de carga, aumentando a resiliência e a disponibilidade da plataforma.

Trade-offs e Riscos: A introdução de camadas intermediárias adiciona uma pequena latência no tempo de resposta (*overhead* de chamadas de métodos e mapeamentos entre objetos de diferentes camadas). Além disso, como o projeto operará sob o Plano Vercel Hobby, assume-se um risco de infraestrutura: a indisponibilidade não advém do esgotamento de *hardware* físico (RAM/CPU), mas do bloqueio automático da plataforma caso o volume transacional ultrapasse as cotas de 100 GB de transferência de dados ou 1 milhão de invocações mensais. O tempo máximo de execução de 10 segundos por função também exige otimização estrita em operações complexas de API, mitigando o estrangulamento.

5.2 Decisão 02: Padrão de Persistência com Data Mapper

Decisão: Isolamento da persistência de dados através do padrão *Data Mapper*, separando completamente as entidades puras do domínio das tabelas e esquemas relacionais do banco de dados PostgreSQL.

Concerns Enquadrados: CC03 (Manutenibilidade e Evolvabilidade) e CC02 (Viabilidade de Construção).

Rationale (Justificativa): Diferente do padrão *Active Record* (onde a lógica de banco de dados fica misturada com a lógica de negócio), o *Data Mapper* permite que as regras complexas do ciclo de vida de pedidos e anúncios do brechó permaneçam agnósticas em relação ao banco de dados. Isso viabiliza a criação de testes unitários robustos e independentes da infraestrutura física, acelerando o ciclo de desenvolvimento em ambiente acadêmico (CC02). Além disso, mudanças na modelagem física do PostgreSQL impactam apenas o arquivo de mapeamento da infraestrutura, protegendo a integridade do núcleo do sistema.

Trade-offs: O isolamento provido pelo *Data Mapper* exige um rigoroso *trade-off* com o Desempenho (CQ01). A separação estrita entre o Domínio e a Infraestrutura adiciona um *overhead* de processamento para realizar o mapeamento e a tradução constante entre os objetos da aplicação e as tabelas relacionais, além de exigir que a equipe escreva um volume maior de código inicial (*boilerplate*).

5.3 Decisão 03: Middleware Orientado a Mensagens (MOM) via WebSockets

Decisão: Adoção de um **middleware** orientado a mensagens (MOM) baseado no protocolo *WebSocket* para estabelecer conexões bidirecionais persistentes (*full-duplex*) entre os clientes e a plataforma. O servidor dedicado, hospedado no serviço PaaS Render, atua como um roteador de mensagens para a comunicação "um-para-um" no subsistema exclusivo de chat interno de usuários.

Concerns Enquadrados: CQ01 (Desempenho — Latência do Chat) e CQ04 (Disponibilidade).

Rationale (Justificativa): Para atender à métrica estrita de entrega de mensagens em até 200ms (CQ01), evitou-se a tática de *HTTP Polling* (que geraria centenas de requisições vazias desnecessárias, sobrecarregando o servidor e degradando o *throughput*). O *WebSocket* mantém um canal aberto com um *handshake* único. Em caso de quedas parciais de rede, a arquitetura implementa uma tática de degradação graciosa na interface do cliente, armazenando as mensagens temporariamente em uma fila local (*local storage*) até o disparo de um evento automático de reconexão do canal, garantindo tolerância a falhas (CQ04). Como funções *serverless* (Vercel) não suportam conexões contínuas, a infraestrutura deste subsistema foi isolada em um serviço especializado para este fim.

Trade-offs e Sensibilidade: A comunicação *full-duplex* é excelente para latência (CQ01), mas manter conexões TCP abertas de forma contínua consome memória RAM do servidor de forma diretamente proporcional ao número de usuários online. Este é um ponto sensível que afeta negativamente a escalabilidade caso não haja limite no número de conexões ativas. Para mitigar esse risco de esgotamento de recursos no PaaS, o escopo das con-

xões ativas deve ser rigorosamente limitado, sendo abertas apenas na tela de negociação de chat, e encerradas automaticamente no desfoque ou inatividade prolongada.

5.4 Decisão 04: Stack Tecnológica de Implementação (Go e React)

Decisão: Adoção do *framework* React para a construção da interface de usuário (SPA) e da linguagem Go (Golang) para o desenvolvimento da API Central e lógicas de *backend*.

Concerns Enquadrados: CQ01 (Desempenho), CQ02 (Escalabilidade) e CC02 (Viabilidade de Construção).

Rationale (Justificativa): Embora a arquitetura lógica do sistema seja agnóstica em relação à tecnologia, a escolha das linguagens de programação visa otimizar os atributos de qualidade no ambiente de implantação escolhido (Vercel). A linguagem Go foi selecionada para o *backend* devido à sua natureza compilada e eficiência em concorrência, o que resulta em um *cold start* extremamente baixo para as *Serverless Functions* da Vercel, maximizando o Desempenho (CQ01) e a Escalabilidade (CQ02). Para a camada de Apresentação, o React permite a componentização da interface e o gerenciamento reativo do estado da sessão (tokens JWT), facilitando o desenvolvimento paralelo pela equipe acadêmica (CC02).

Trade-offs: A segregação de tecnologias (JavaScript/TypeScript no *frontend* e Go no *backend*) impede o reuso de código (como tipagens e validações de domínio) entre as pontas, exigindo maior rigor na manutenção dos contratos de API.

6 Correspondences e Correspondence Rules

Conforme estabelecido pela ISO/IEC/IEEE 42010:2022, as *Correspondences* (Correspondências) expressam as relações entre os elementos arquiteturais presentes em diferentes *Architecture Views*, enquanto as *Correspondence Rules* (Regras de Correspondência) definem as restrições e invariantes que governam essas relações.

O objetivo desta seção é garantir a consistência global e auditar a *Architecture Description*, assegurando rastreabilidade de ponta a ponta: desde as exigências de negócio e restrições de qualidade (Seção 2), passando pelas decisões técnicas tomadas (Seção 5), até a materialização nos artefatos de código e topologia física (Seção 4). Isso previne a erosão arquitetural durante o ciclo de desenvolvimento.

6.1 Regras de Correspondência (Correspondence Rules)

As seguintes regras normativas foram estipuladas para governar a arquitetura do Sistema de Intermediação de Brechós.

- **CR-01 (Completeness e Alocação de Implantação):** Todo componente de software lógico mapeado e estruturado nos viewpoints deve possuir rastreabilidade exata para pelo menos um nó de execução no Diagrama de Implantação (VP4). Nenhum artefato de software pode existir flutuando sem um contêiner ou ambiente de hospedagem físico associado.

- **CR-02 (Cobertura de Atributos de Qualidade):** Todo concern de qualidade priorizado (CQ01 a CQ04) deve estar fundamentado e mitigado por, no mínimo, uma Decisão Arquitetural documentada (Seção 5). *Validação: Matriz de Rastreabilidade Qualidade-Decisão (Seção 6.2.1).*
- **CR-03 (Isolamento de Domínio e Inversão de Controle):** No Viewpoint de Desenvolvimento (VP2), os artefatos contidos na camada de **Domínio** (regras de intermediação C2C, ciclo de vida de pedidos) estão expressamente proibidos de acoplar dependências externas (APIs HTTP, conectores PostgreSQL). A injeção de dependência deve ser utilizada para inverter o controle, forçando a camada de Infraestrutura a implementar as interfaces definidas pelo Domínio. *Validação: Revisão de código (Pull Requests) garantindo a ausência de imports de infraestrutura nas classes de entidade.*
- **CR-04 (Delegação Financeira Restrita):** Nenhum componente interno do sistema está autorizado a persistir ou trafegar dados completos de cartões de crédito (PAN, CVV) no banco de dados da aplicação. O fluxo deve empregar *tokens* opacos gerados no *frontend* e enviados diretamente ao *Gateway* de Pagamento. *Validação: Auditoria no mapeamento físico do banco de dados (esquema SQL).*

6.2 Correspondências Identificadas (Correspondences)

As matrizes a seguir formalizam a aplicação prática das regras definidas, evidenciando a consistência e a integridade desta *Architecture Description*.

6.2.1 Mapeamento de Concerns para Decisões Arquiteturais (Aplica CR-02)

Esta matriz comprova que as restrições não funcionais do sistema foram ativamente resolvidas através de táticas arquiteturais concretas.

ID / Concern	Atributo Alvo	Decisão Arquitetural e Tática de Mitigação
CQ01	Desempenho	Decisão 01, 03 e 04: Delegação de processamento assíncrono, uso estrito de conexões persistentes (<i>WebSockets</i>) para reduzir o <i>overhead</i> no <i>chat</i> , e adoção de Go para minimizar latência de inicialização (<i>cold starts</i>).
CQ02	Escalabilidade	Decisão 01 e 04: Adoção de arquitetura inerentemente <i>stateless</i> via <i>Serverless Functions</i> da Vercel (potencializada pela eficiência da linguagem Go), com controle de sessão via JWT. A plataforma escala transparentemente de zero a milhares de funções concorrentes sob demanda.
CQ03	Segurança	Decisão 01 e CR-04: Isolamento de autenticação via tokens e não-armazenamento de dados sensíveis de pagamento (delegação ao Stripe).
CQ04	Disponibilidade	Decisão 03: Degradação graciosa no <i>frontend</i> , com tolerância a falhas na fila de mensagens locais (<i>local storage</i>) durante interrupções de conexão.

ID / Concern	Atributo Alvo	Decisão Arquitetural e Tática de Mitigação
CC03	Manutenibilidade	Decisão 02 e CR-03: Uso rigoroso do padrão <i>Data Mapper</i> . Permite que a lógica central do brechó seja testada de forma unitária, acelerando a identificação de falhas e refatorações.

6.2.2 Rastreabilidade Lógica vs. Física de Implantação (Aplica CR-01)

A matriz a seguir demonstra a alocação dos elementos lógicos nos respectivos nós de infraestrutura necessários para a operação do sistema web.

Elemento Lógico	Nó de Execução (VP4)	Relação Semântica
App Cliente (Web SPA)	Navegador Web do Usuário final	<i>Is executed by</i> (Instanciado e interpretado localmente via <i>browser</i>)
Serviço de Arquivos Estáticos	Vercel Edge Network (CDN)	<i>Is hosted on</i> (Responsável por entregar <i>assets</i> , HTML, JS empacotados)
API Central (Regras C2C)	Vercel Serverless Functions	<i>Is deployed on</i> (Processa as transações de compra e edição de anúncios)
Subsistema de <i>Chat</i>	Servidor de <i>WebSocket</i> em PaaS Externo (Render)	<i>Is deployed on</i> (Mantém as conexões TCP abertas e gerencia o <i>broadcasting</i>)
Módulo de Persistência	<i>Cluster</i> PostgreSQL Gerenciado	<i>Reads/Writes to</i> (Garante o armazenamento permanente e ACID das entidades)
Gerenciador de Mídias	Amazon S3 <i>Bucket</i>	<i>Communicates with</i> (Armazena e distribui os binários pesados de imagens)

6.2.3 Matriz de Restrição de Dependência de Camadas (Aplica CR-03)

Para garantir que a manutenibilidade não seja corrompida por acoplamento indevido, a arquitetura estabelece um vetor direcional estrito de dependências. A tabela abaixo mapeia as relações permitidas entre as camadas do *backend*.

Camada Origem	Pode depender e importar recursos da(s) Camada(s) Destino
Apresentação (Controladores Rest)	Camada de Aplicação (Casos de Uso)
Aplicação (Casos de Uso)	Camada de Domínio (Entidades e Interfaces)
Infraestrutura (Repositórios)	Camada de Domínio (Implementa as Interfaces do Domínio)

Camada Origem	Pode depender e importar recursos da(s) Camada(s) Destino
Domínio (Regras de Negócio C2C)	NENHUMA. (O Domínio é puro e isolado de todas as outras camadas).